

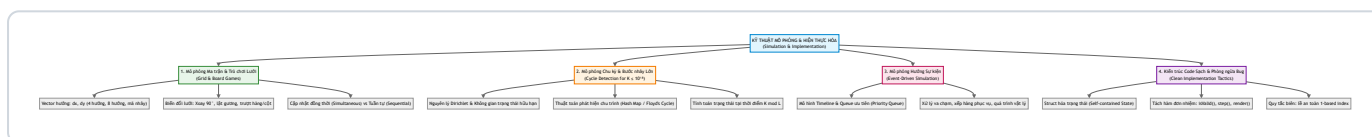
Chuyên đề: Kỹ thuật Mô phỏng & Hiện thực hóa Giải thuật (Simulation & Implementation Techniques)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Sơ đồ Tư duy & Kiến trúc Chuyên đề



1. Bản chất của Bài toán Mô phỏng trong Lập trình Thi đấu

Trong các kỳ thi như **ICPC**, **Olympic Tin học (OLP)**, **Codeforces (Div 2/3)** và **AtCoder**, dạng bài **Mô phỏng (Simulation)** chiếm tỷ lệ rất cao ở các bài khởi động hoặc phân loại trung cấp.

Khác với các bài toán quy hoạch động hay cấu trúc dữ liệu thuần túy (vốn đòi hỏi phát hiện quy luật trừu tượng), đề bài mô phỏng thường:

- Rất dài và nhiều quy tắc chi tiết:** Mô tả một hệ thống thực tế (di chuyển của robot, luật chơi cờ, xử lý đơn hàng, quá trình phân rã hạt, v.v.).
- Không đòi hỏi thuật toán phức tạp:** Thuật toán chỉ đơn giản là *làm chính xác những gì đề bài yêu cầu từng bước một*.
- Là "hố chôn thời gian" (Time Sink):** Thí sinh dễ bị lạc vào mê cung các câu lệnh `if-else` chằng chịt, dẫn đến lỗi biên (off-by-one), code rối rắm không debug nổi, hoặc nộp bài dính `Wrong Answer` ở các test ẩn khó phát hiện.

Khẩu quyết Simulation:

"Đừng vội gõ code khi chưa mô hình hóa được trạng thái (State) và luật chuyển trạng thái (Transition). Code mô phỏng phải viết theo kiểu module hóa tuyệt đối: ngắn gọn, rõ ràng, dễ test từng hàm."

2. Kỹ thuật Xử lý Lưới 2D (Grid & Board Simulation)

Hơn 60% các bài toán mô phỏng diễn ra trên bảng chữ nhật kích thước $M \times N$.

2.1. Vector Hướng Di chuyển (dx, dy)

Thay vì viết 4 khối `if (dir == 'U') ... else if (dir == 'R') ...`, ta luôn mã hóa hướng bằng các mảng hằng số:

```
// 4 hướng trực giao: U, R, D, L (theo chiều kim đồng hồ)
const int dx[4] = {-1, 0, 1, 0};
const int dy[4] = {0, 1, 0, -1};
const char dirChar[4] = {'U', 'R', 'D', 'L'};

// 8 hướng (kèm đường chéo):
const int dx8[8] = {-1, -1, -1, 0, 0, 1, 1, 1};
const int dy8[8] = {-1, 0, 1, -1, 1, -1, 0, 1};

// 8 nước đi của quân Mã (Knight moves):
const int kx[8] = {-2, -2, -1, -1, 1, 1, 2, 2};
const int ky[8] = {-1, 1, -2, 2, -2, 2, -1, 1};
```

Xoay hướng linh hoạt bằng Modulo:

- Rẽ phải 90° : `dir = (dir + 1) % 4;`
- Rẽ trái 90° : `dir = (dir + 3) % 4;`
- Quay ngược 180° : `dir = (dir + 2) % 4;`

Hàm kiểm tra biên an toàn:

```
inline bool isValid(int r, int c, int n, int m) {
    return (r >= 0 && r < n && c >= 0 && c < m);
}
```

2.2. Biến đổi Ma trận: Phép Xoay & Lật (Rotation & Reflection)

Nhiều bài toán yêu cầu xoay bảng cờ hoặc kiểm tra tính đối xứng của hình dạng:

Phép biến đổi	Kích thước mới	Tọa độ mới (r' , c') từ (r , c) cũ
Xoay 90° theo chiều kim đồng hồ	$M \times N$	$r' = c, \quad c' = N - 1 - r$
Xoay 180°	$N \times M$	$r' = N - 1 - r, \quad c' = M - 1 - c$
Xoay 270° (tức 90° ngược chiều KĐH)	$M \times N$	$r' = M - 1 - c, \quad c' = r$
Lật gương theo trục ngang (Horizontal Flip)	$N \times M$	$r' = N - 1 - r, \quad c' = c$
Lật gương theo trục dọc (Vertical Flip)	$N \times M$	$r' = r, \quad c' = M - 1 - c$

```
// Hàm xoay bảng 90 độ theo chiều kim đồng hồ
vector<string> rotate90(const vector<string>& grid) {
    int n = grid.size(), m = grid[0].size();
    vector<string> res(m, string(n, ' '));
    for (int r = 0; r < n; ++r) {
        for (int c = 0; c < m; ++c) {
            res[c][n - 1 - r] = grid[r][c];
        }
    }
    return res;
}
```

2.3. Cạm bẫy: Cập nhật Đồng thời (Simultaneous) vs Tuần tự (Sequential)

Trong các bài như **Conway's Game of Life**, sự sinh/tử của một ô ở thế hệ $t + 1$ phụ thuộc vào trạng thái của các ô lân cận ở thế hệ t .

⚠ **Cạm bẫy chết người:** Cập nhật trực tiếp lên bảng hiện tại `grid[r][c] = newVal` sẽ làm sai lệch thông tin của các ô lân cận được xét sau đó!

```
// ❌ SAI LẦM: Ghi đè trực tiếp
for (int r = 0; r < n; ++r) {
    for (int c = 0; c < m; ++c) {
        grid[r][c] = computeNextState(r, c, grid); // LÀM HỒNG TRẠNG THÁI CŨ!
    }
}

// ✅ ĐÚNG ĐẮN: Dùng bảng phụ (Double Buffering)
vector<string> nextGrid = grid;
for (int r = 0; r < n; ++r) {
    for (int c = 0; c < m; ++c) {
        nextGrid[r][c] = computeNextState(r, c, grid); // Đọc từ grid cũ
    }
}
grid = move(nextGrid); // Hoán đổi bảng O(1)
```

3. Kỹ thuật Mô phỏng Chu kỳ với Bước nhảy Cực lớn ($K \leq 10^{18}$)

3.1. Bản chất: Nguyên lý Dirichlet & Không gian Trạng thái Hữu hạn

Giả sử hệ thống chuyển trạng thái tất định $S_{t+1} = f(S_t)$.

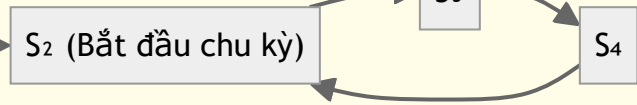
Nếu số lượng trạng thái phân biệt có thể có là U (với $U \leq 10^6$), thì theo **Nguyên lý Chuồng bồ câu (Pigeonhole Principle)**:

Chắc chắn sau tối đa U bước, một trạng thái nào đó sẽ lặp lại, tạo thành một **chu kỳ (Cycle)**.

Đoạn mở đầu (Pre-cycle tail): Độ dài = λ



Vòng lặp tuần hoàn (Cycle Loop): Chu kỳ = L



Giả sử:

- Chu kỳ xuất hiện lần đầu ở bước thứ λ (S_λ).
- Trạng thái này lặp lại lần tiếp theo ở bước μ .
- Độ dài chu kỳ tuần hoàn là $L = \mu - \lambda$.

Khi cần tính trạng thái tại bước K cực lớn ($K \leq 10^{18}$):

1. Nếu $K < \lambda$: Trạng thái đơn giản là S_K .
2. Nếu $K \geq \lambda$: Chỉ cần lấy phần dư modulo:

$$K' = \lambda + (K - \lambda) \pmod{L}$$

Trạng thái tại bước K đồng nhất với trạng thái tại bước K' . Độ phức tạp giảm từ $O(K)$ xuống $O(U)$!

3.2. Cài đặt Tổng quát bằng Bảng băm hoặc Mảng đánh dấu

```
struct SimulationCycleSolver {
    // Lưu lịch sử các trạng thái để truy xuất O(1)
    vector<State> history;
    // Map từ hash/biểu diễn trạng thái sang chỉ số bước xuất hiện đầu tiên
    map<State, int> visitedStep;

    State solve(State startState, long long K) {
        State cur = startState;
        history.push_back(cur);
        visitedStep[cur] = 0;

        for (long long step = 1; step <= K; ++step) {
            cur = getNextState(cur);

            if (visitedStep.count(cur)) {
                // Đã phát hiện chu kỳ lặp!
                int firstSeen = visitedStep[cur];
                int cycleLength = step - firstSeen;

                long long remaining = K - step;
                long long finalIdx = firstSeen + (remaining % cycleLength);
                return history[finalIdx];
            }

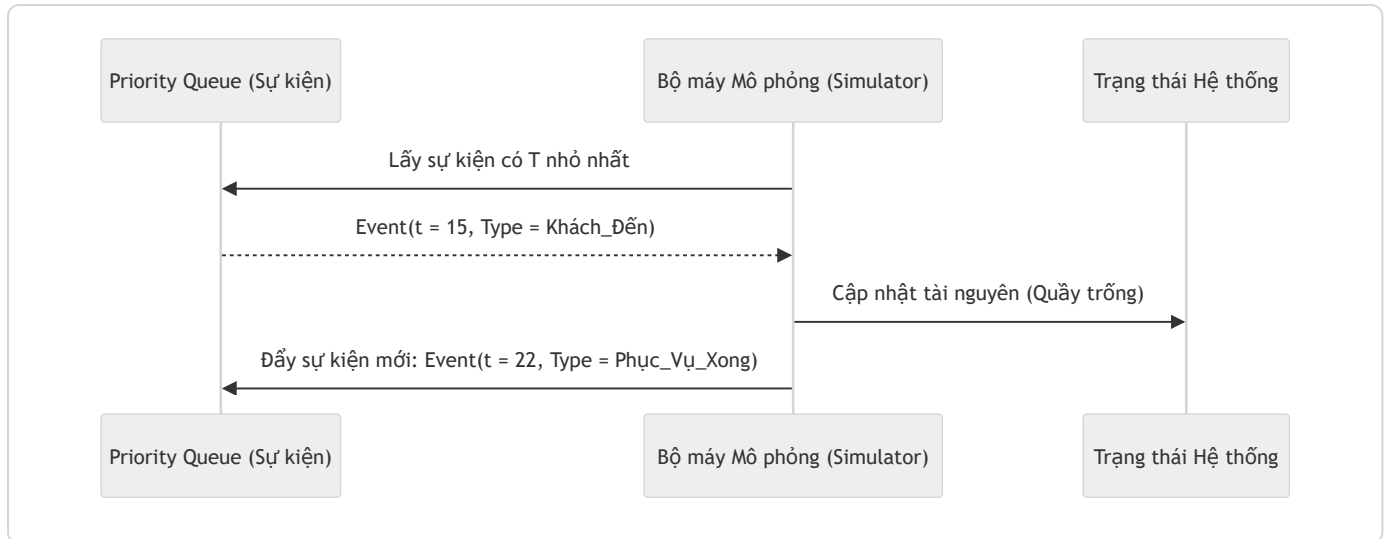
            visitedStep[cur] = step;
            history.push_back(cur);
        }

        return cur;
    }
};
```

4. Mô phỏng Hướng Sự kiện (Event-Driven Simulation)

Thay vì tăng dần từng đơn vị thời gian $\Delta t = 1$ (sẽ bị TLE nếu thời gian lên đến 10^9 và hầu hết các mốc thời gian không có gì xảy ra), ta dùng mô hình **Bước nhảy Sự kiện**:

1. Đưa các sự kiện tương lai vào một **Hàng đợi ưu tiên (Priority Queue)**, sắp xếp theo thời gian xảy ra T nhỏ nhất.
2. Tại mỗi bước, rút sự kiện sớm nhất ra xử lý.
3. Sự kiện này có thể sinh ra các sự kiện mới trong tương lai và đẩy ngược vào Priority Queue.



Ví dụ kinh điển: Hệ thống Phục vụ Khách hàng tại Ngân hàng

Mỗi khách đến vào thời điểm A_i và cần thời gian phục vụ D_i trên K quầy giao dịch.

Hàng đợi ưu tiên `priority_queue<long long, vector<long long>, greater<long long>> counters` lưu thời điểm rảnh rồi sớm nhất của từng quầy. Độ phức tạp xử lý N khách hàng chỉ là $O(N \log K)$ thay vì lặp qua từng giây.

5. Chiến lược Hiện thực hóa Code Sạch (Clean Implementation Tactics)

Trong phòng thi, một bài mô phỏng phức tạp có thể dài từ 150 đến 300 dòng code. Để tránh biến code thành một "bãi rác logic" không thể cứu vãn:

1. Đóng gói dữ liệu bằng `struct` :

```

```cpp
struct Robot {
 int x, y;
 int dir; // 0: U, 1: R, 2: D, 3: L
 bool isAlive;

 void turnLeft() { dir = (dir + 3) % 4; }
 void turnRight() { dir = (dir + 1) % 4; }
 void stepForward() { x += dx[dir]; y += dy[dir]; }
};

```

2. **Quy tắc Hàm Đơn nhiệm (Single Responsibility Principle)**: - Viết hàm `parseInput()` , `simulateOneStep()` , `checkEndCondition()` , `printResult()` . - Hàm `main()` chỉ đóng vai trò nhạc trưởng điều phối vòng lặp. 3. **Trace bằng ASCII/Debug Output cục bộ**: - Viết sẵn hàm `debugPrintBoard()` được bọc trong cờ `#ifndef ONLINE_JUDGE` để trực quan hóa bàn cờ ra màn hình console khi chạy thử test mẫu.

## 6. Case Study Thực chiến

### Case Study: Robot Di chuyển trên Lưới có Chướng ngại vật & Lệnh Lặp Lớn

**Bài toán:** Một robot xuất phát tại tọa độ  $(0, 0)$  hướng Bắc trên bảng  $N \times M$ . Nhận chuỗi lệnh độ dài  $L$  gồm **F** (tiến 1 bước), **L** (quay trái  $90^\circ$ ), **R** (quay phải  $90^\circ$ ). Chuỗi lệnh này được lặp lại liên tục  $K$  lần ( $K \leq 10^9$ ). Nếu bước tiếp theo là ô chướng ngại vật **#** hoặc vượt biên, robot đứng yên tại chỗ và bỏ qua lệnh **F** đó. Tìm tọa độ cuối cùng của robot.

**Phân tích thuật toán:**

- Vì bảng có kích thước  $N, M \leq 50$ , robot có 4 hướng mặt, và chuỗi lệnh có độ dài  $L \leq 100$ .
- Không gian trạng thái phân biệt tại đầu mỗi chu kỳ lệnh là:

$$\text{Số trạng thái} \leq N \times M \times 4 = 50 \times 50 \times 4 = 10\,000$$

- Vì  $10\,000 \ll 10^9$ , theo Dirichlet, chắc chắn sẽ xuất hiện chu kỳ lặp lại ở đầu một chu kỳ lệnh nào đó trong vòng  $\leq 10\,000$  vòng lặp!
- Ta chỉ cần mô phỏng từng chuỗi lệnh  $L$ , ghi nhớ bộ ba  $(x, y, dir)$  sau mỗi lần hoàn thành chuỗi lệnh. Khi gặp lại trạng thái cũ, lập tức tính modulo  $K$  để nhảy tới kết quả cuối cùng trong chớp mắt.

## 7. Cạm bẫy Thường gặp & Danh mục Kiểm tra Trước khi Nộp bài

STT	Cạm bẫy phổ biến	Giải pháp khắc phục
1	<b>Nhầm lẫn giữa trục Tọa độ Decartes <math>(x, y)</math> và Ma trận <math>(row, col)</math></b>	Quy ước thống nhất ngay từ đầu: $r$ tăng là đi xuống (hàng), $c$ tăng là đi sang phải (cột). $x \leftrightarrow c, y \leftrightarrow r$ .
2	<b>Cập nhật dữ liệu dạng "Đồng thời" nhưng lại ghi đè "Tuần tự"</b>	Sử dụng 2 mảng (Double Buffering) <code>cur</code> và <code>next</code> , chỉ cập nhật vào <code>next</code> dựa trên giá trị đọc từ <code>cur</code> .
3	<b>Tràn số với tham số thời gian <math>K</math></b>	Luôn dùng <code>long long</code> cho số bước lặp, mốc thời gian sự kiện, và các phép tính modulo chu kỳ.
4	<b>Bỏ sót lệnh sau khi chu kỳ kết thúc</b>	Khi modulo $K$ , hãy cẩn thận tính đúng chỉ số bắt đầu của chu kỳ ( $\lambda$ ) và độ dài chu kỳ ( $L$ ).
5	<b>Bộ nhớ băm trạng thái quá nặng</b>	Nén trạng thái $(r, c, dir)$ thành một số nguyên duy nhất: <code>int id = (r * m + c) * 4 + dir;</code> để dùng mảng đánh dấu $O(1)$ thay cho <code>std::map</code> .



## **Competitive Programming Handbook**

*SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)*

*Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool*

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.